

Minkowski_eng vs TorchSparse 1.4 using SegmentedForests modified dataset

epochs: 100

num_workers: 6

batch_size: 16

GPU: NVIDIA A100 80GB

CPU: AMD EPYC 7713 64-Core

RAM: 256GB

Runtime:

2d 2h 50m 23s for MinkowskiEng

3d 21h 11m 6s for TorchSparse 1.4

Project realized as the final assignment of the Subject:

Introduction to AI in Forestry supervised by Ph.D. Maciej Wielgosz

a part of "Geomatics and AI in Forestry" Postgraduate Studies realized by
Forest Research Institute (Poland)

Jakub Wolosz

Last updated: September 22, 2026



✓ Aim of the project

The aim of the project was narrow and deliberately falsifiable: take **ForAINet**, a published deep-learning model for forest point-cloud segmentation, train it on a dataset it was never designed for, then swap its sparse-convolution backbone from MinkowskiEngine to TorchSparse 1.4, retrain, and compare the two models against each other and against the figures reported in the original publication. Everything below serves that comparison.

✓ Methodology

✓ Used materials

✓ The dataset

The input is **SegmentedForests** — fourteen **terrestrial** LiDAR forest plots, published open-access on [Zenodo](#) under an MIT licence and described in *Forestry* ([10.1093/forestry/cpaf062](#)). It is the only permanent input to the whole project: 4.4 GB of compressed LAZ, and **around 800 M labelled points** as prepared for training.

Plots were assigned to splits by point count, holding out the smallest so that the bulk of the data stays in training:

Split	Plots	Points
train	11 plots	787,235,845
validation	plot_11	18,945,417
test	plot_01, plot_14	41,534,130

Only the two **test** plots are used for the headline results; plot_11 is excluded from them because training selected its checkpoints on it.

✓ Unifying the data into a form ForAINet accepts

The dataset and the model do not speak the same language, and four separate translations were needed.

None of them are cosmetic — each one, left undone, produces a model that trains without complaint and learns the wrong thing.

Labels: sixteen source classes into four. The dataset's `Class` field takes sixteen distinct values (verified as the exact union across all fourteen plots), while ForAINet expects a field called `semantic_seg`. The map merges them:

Source Class	→	Meaning
8, 9, 11	0	unclassified — ignored by the loss, but kept in the cloud
0, 4, 5, 6, 7, 12, 13, 22, 23	1	low vegetation
1	2	ground
3, 10	3	stem points
2	4	live branches

The map is deliberately **total**: a source value missing from it stops that plot with an error rather than being guessed at, because a guess would smuggle a meaningless class into training. The *unclassified* points are not deleted — ForAINet's loss ignores them, but their geometry still supports the features of every point around them, and removing them would punch holes in the cloud.

Tree identifiers: cleared everywhere except on trees. ForAINet's convention is that anything which is not part of a tree carries instance id `0`. The instance-segmentation tool used to produce the identifiers (3DFin) follows the opposite convention — it stamps every point with its nearest stem's id, ground included. ForAINet's own filter then discards any instance whose id also appears on a non-tree point, so with the raw ids **every tree was discarded**: zero of forty-two instances survived on one test plot. Clearing the id outside the two tree classes touched **395,410,648 points — 46.6 % of the dataset** — and restored all forty-two.

Coordinates centred. Each plot has its own minimum subtracted so that all coordinates are non-negative. The shifts are recorded per plot so that predictions can be put back into the original coordinate system afterwards.

The split written into the file name. ForAINet has no split manifest at all; it reads the set from the filename, testing whether the characters before the extension spell `val` or `test`. Without the suffix, all fourteen plots would silently have become training data.

✓ Where the instance labels come from

SegmentedForests labels every point with a semantic class, but it contains no tree identifiers — its published purpose is semantic segmentation, and the only annotation columns in the source files are the class and a split marker. ForAINet, however, is a panoptic model: it needs both a class for every point and an instance id for every individual tree.

The instance labels therefore had to be generated. 3DFin 0.6.0, an open-source forest inventory tool, runs over every plot as the first stage of the pipeline and assigns each point the id of the tree it belongs to. Only then can the preparation stage run at all — it reads 3DFin's output, never the raw clouds.

This is a limitation worth stating up front rather than burying in the results: the instance ground truth in this project is algorithmically derived, not annotated by hand, whereas the publication's instance results are measured against manually delineated trees. The semantic labels are unaffected — those are the dataset's own.

✓ How this dataset differs from the publication's

This is the single most important difference in the project, and it frames every result.

ForAINet was built and tuned for **high-density airborne** LiDAR — the FOR-Instance dataset, flown over the canopy. SegmentedForests is **terrestrial**: the scanner sits on the forest floor. The two see almost inverted worlds. From above, crowns are well sampled and stems are sparse; from below, stems are dense and crowns are occluded and interleaved with their neighbours'. Point-density profiles, occlusion patterns and the very geometry that separates one tree from the next are all different.

There is also one fewer class. Upstream's fifth class, `branches`, has no counterpart in SegmentedForests, so rather than leave the model with an output that no label could ever reach, the class was removed from the framework — a change described in section 3.

✓ Scripts written for the project

Script	Role
<code>main_pipeline.py + pipeline/chain.py</code>	runs the whole preparation per plot, deleting each intermediate the moment it is consumed
<code>pipeline/threedfin.py</code>	batch-drives the 3DFin instance-segmentation tool over every plot
<code>forainet_prep.py</code>	the label mapping, id clearing, centring and split naming; also restores predictions to real-world coordinates
<code>pipeline/convert.py</code>	LAZ ↔ PLY conversion, written with no heavyweight dependencies so it also runs inside the training container
<code>class_unifier.py</code>	diagnostic export keeping the original and remapped labels side by side, for checking that a merge did what was intended
<code>pipeline/parallel.py</code>	runs several plots at once, bounded by available memory rather than by CPU count
<code>misc/check_forainet_classes.py</code>	re-derives what the framework should contain from my own configuration and verifies it — the guard against silently losing the patch
<code>container_export/preflight.sh + smoke_test.py</code>	a one-command pass/fail gate run on a cheap GPU before renting an expensive one
<code>misc/view_split_point_cloud.ipynb + view_cloud_3d.py</code>	a two-pane point-cloud viewer used to inspect label merges visually

✓ Used technologies and software

Role	What was used
Data preparation (local)	Python 3.13, Hydra / hydra-zen for configuration, laspy + lazrs, plyfile, NumPy, PyYAML, 3DFin 0.6.0

Role	What was used
Visual inspection	matplotlib, tkinter, PyVista + VTK
The model	PyTorch, torch-points3d, ForAINet (pinned at commit 5fe600a), MinkowskiEngine 0.5.x, TorchSparse 1.4
Infrastructure	Docker Desktop (WSL2), Docker Hub, Cloudflare R2, Vast.ai (NVIDIA A100 80 GB)
Experiment tracking	Weights & Biases
Development	git with submodules, Claude Code as the coding assistant throughout

✓ Encapsulating the training environment in a container

Training had to move to a rented GPU, which means the entire environment has to travel. The steps were:

1. **Start from the existing `for-ai-net` base image rather than rebuilding it.** Several of its layers are no longer reproducible — MinkowskiEngine compiled from git master, TorchSparse, a dependency installed from a zip archive. Rebuilding risks never getting that environment back. It ships Python 3.8.10 and CUDA kernels compiled for compute capabilities 6.0 through 8.6, with no fallback for anything newer.
2. **Add one thin layer on top**, a purpose-built `Dockerfile.train`. It installs a LAZ backend, because the base image's laspy cannot physically open the dataset; pins NumPy to the version already present so that the package resolver cannot drag the PyTorch stack sideways; and copies the format converter to a path outside the working directory, which must stay an exact mirror of the local code.
3. **Make the build fail early.** A smoke test runs as a build step: it writes a two-point LAZ file carrying the label fields, reads it back, and asserts the labels survived. A broken reader is then caught on a laptop in seconds instead of on a rented GPU after the data has already downloaded.

4. **Publish the pieces separately** — the image to Docker Hub, and 4.32 GB of compressed point clouds to Cloudflare R2, chosen because it charges no egress fee for a file that will be downloaded repeatedly.
5. **Choose the hardware by compute capability, not by price.** The A100 is 8.0 and supported; a newer RTX 4090, L40S or H100 would have no compiled kernels and no just-in-time fallback, so it could not execute a single convolution.
6. **Gate the instance before committing to a long run.** The pre-flight script locates or clones the framework at the pinned commit, applies the local patch, checks that the configuration files parse under the *container's* library versions, expands the point clouds, and runs two epochs of each backbone.

One consequence is worth stating plainly: the image contains the *environment*, not the code. Locally the code arrives through a folder mounted from the host, which a rented machine cannot have — which is precisely why the pre-flight script clones it.

✓ Changes to the original software

ForAINet is included here as a **git submodule**, and a submodule records nothing but a commit hash. Every local edit is therefore invisible to this project's history and is reverted by a routine `git submodule update` — without anything crashing. The edits are kept instead as a patch file, `patches/forainet-local.patch`, covering **twelve files**, and a checker script re-derives the expected contents from my own configuration and verifies them in twenty-one separate assertions.

Grouped by purpose:

Purpose	What changed
Five classes to four	the class tables (which the framework declares twice), the valid-class and instance-class lists, and the counters in both evaluation routines. Upstream's <code>branches</code> class has no counterpart in this dataset
Making TorchSparse usable	a new model file selected by a single configuration switch, plus two fixes in the TorchSparse compatibility layer. Without them that backbone crashes — it never records the coordinate and kernel maps its scoring sub-network needs, because that sub-network never operates at the input resolution
Correctness and runnability	replacing NumPy aliases removed in version 1.24; disabling a pre-trained checkpoint path that pointed at a <i>five-class</i> model on the original author's cluster, which would have been loaded silently and partially; making the evaluation configuration runnable at all; and five repairs to the final pooled-report script, which as shipped could not execute
Performance	replacing a worker pool that was created and destroyed on every forward pass — roughly 196 times per epoch — with a single shared one; and switching the three evaluation output writers from text to binary, which took a three-plot evaluation from 1 h 52 min to about 12 minutes

Reapplying the patch is one command, but **verifying it is the part that matters**. The obvious check — asking git whether the patch reverses cleanly — is *necessary but not sufficient*: it only compares the context around each change, so an edit lying outside every one of them passes unnoticed. That happened during the project. The reliable test rebuilds a pristine copy of the framework from the patch and compares every file it touches.

✓ Results

I trained two models: the same ForAINet network on the same data with the same settings on the same machine, differing **only** in the sparse-convolution library underneath — MinkowskiEngine (the original) and TorchSparse 1.4. Both ran for 99 epochs on a rented NVIDIA A100 80 GB. Everything below is measured; nothing is estimated.

✓ Training

Each chart carries two lines, one per backbone, and the first thing to notice is how hard they are to tell apart. The semantic loss falls steeply over the first few epochs and then flattens; validation mIoU climbs to roughly 65 and hovers there — for **both** models, within a fraction of a point of each other all the way to the end:

Tracked during training (subsamped) MinkowskiEngine TorchSparse 1.4

Final validation mIoU	65.09	64.99
Final test mIoU	70.03	69.71
Final test F1	0.486	0.468

That near-identity is itself a result: swapping the sparse-convolution library changed how fast the network trained, not what it learned.

The second thing to notice is that the instance metrics — mean average precision, mean recall, the offset and score losses — sit flat at zero for the first thirty epochs on both lines, and only come alive at epoch 31. That is by design. A setting called

`prepare_epoch: 30` gates the entire instance branch: for thirty epochs the network learns only to classify points, and only then does it begin grouping them into individual trees. The side effect is expensive, and section 4 returns to it — a large part of the codebase does not execute until thirty epochs of paid GPU time have been spent.

It also has a practical consequence visible in the curves. TorchSparse's best validation mIoU falls at **epoch 22**, before the instance branch had switched on at all, so that checkpoint's tree-scoring network was still untrained. The final weights were therefore used for both models rather than each one's nominal best.

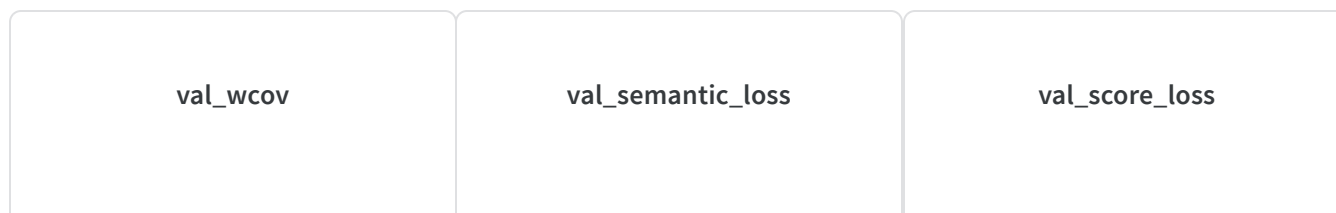
The per-epoch cost is the clearest speed result the project produced:

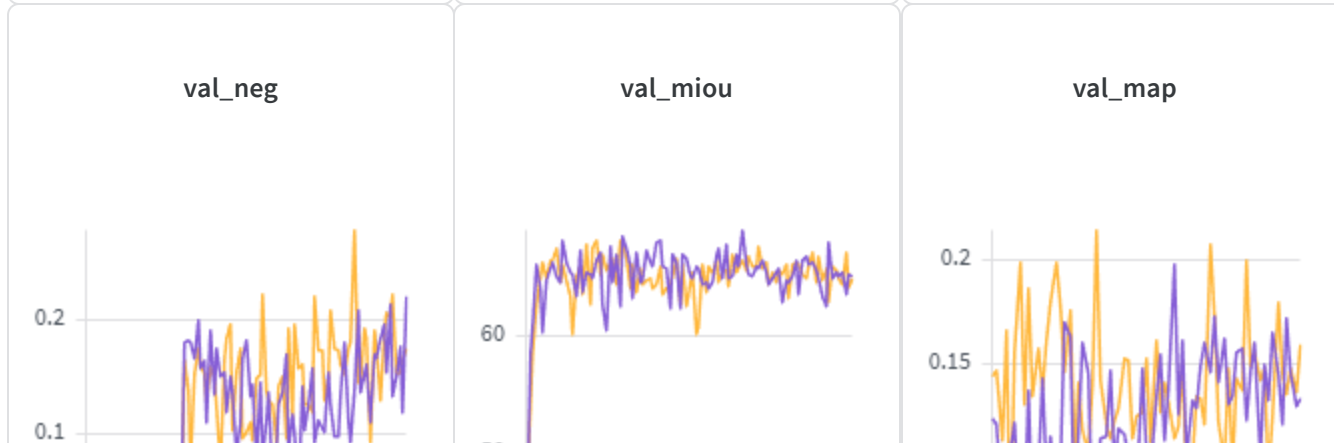
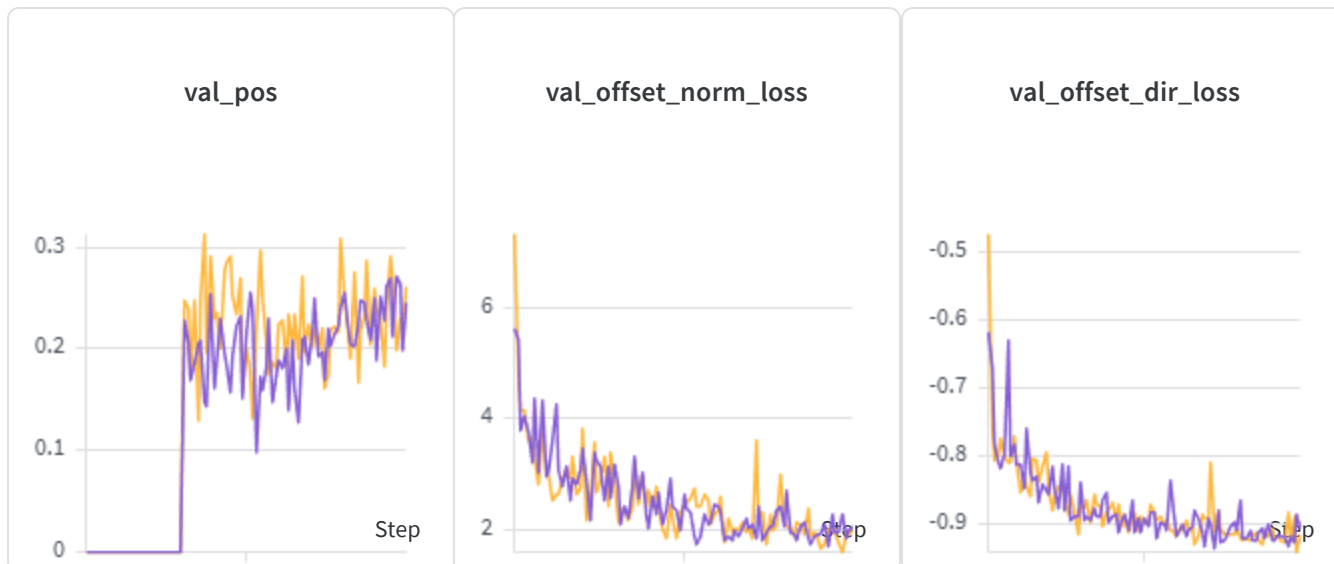
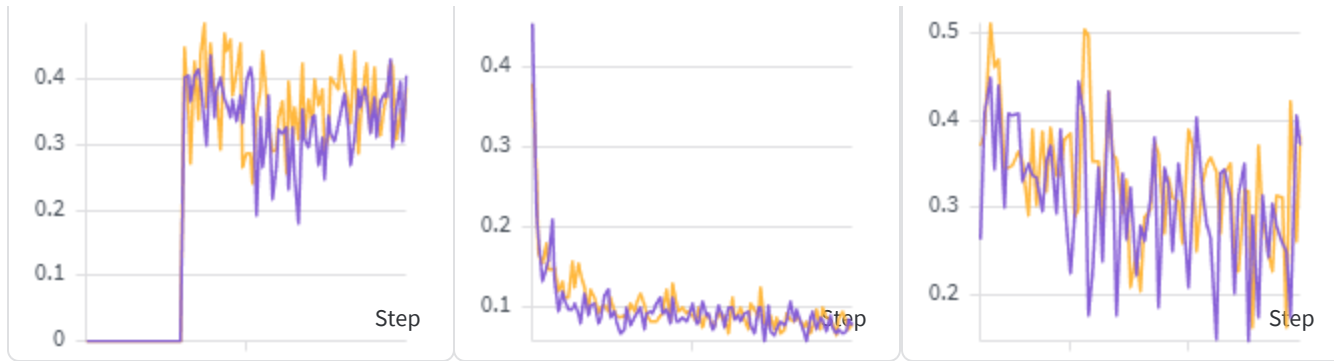
Phase	MinkowskiEngine	TorchSparse 1.4
Epochs 1–30 (backbone and heads only)	1.04 s/iteration · 3 min 16 s per epoch	1.26 s/iteration · 3 min 57 s per epoch
Epoch 31 onward (instance scorer active)	13.70 s/iteration · ~44 min per epoch	27.8–29.6 s/iteration · ~1 h 32 min per epoch

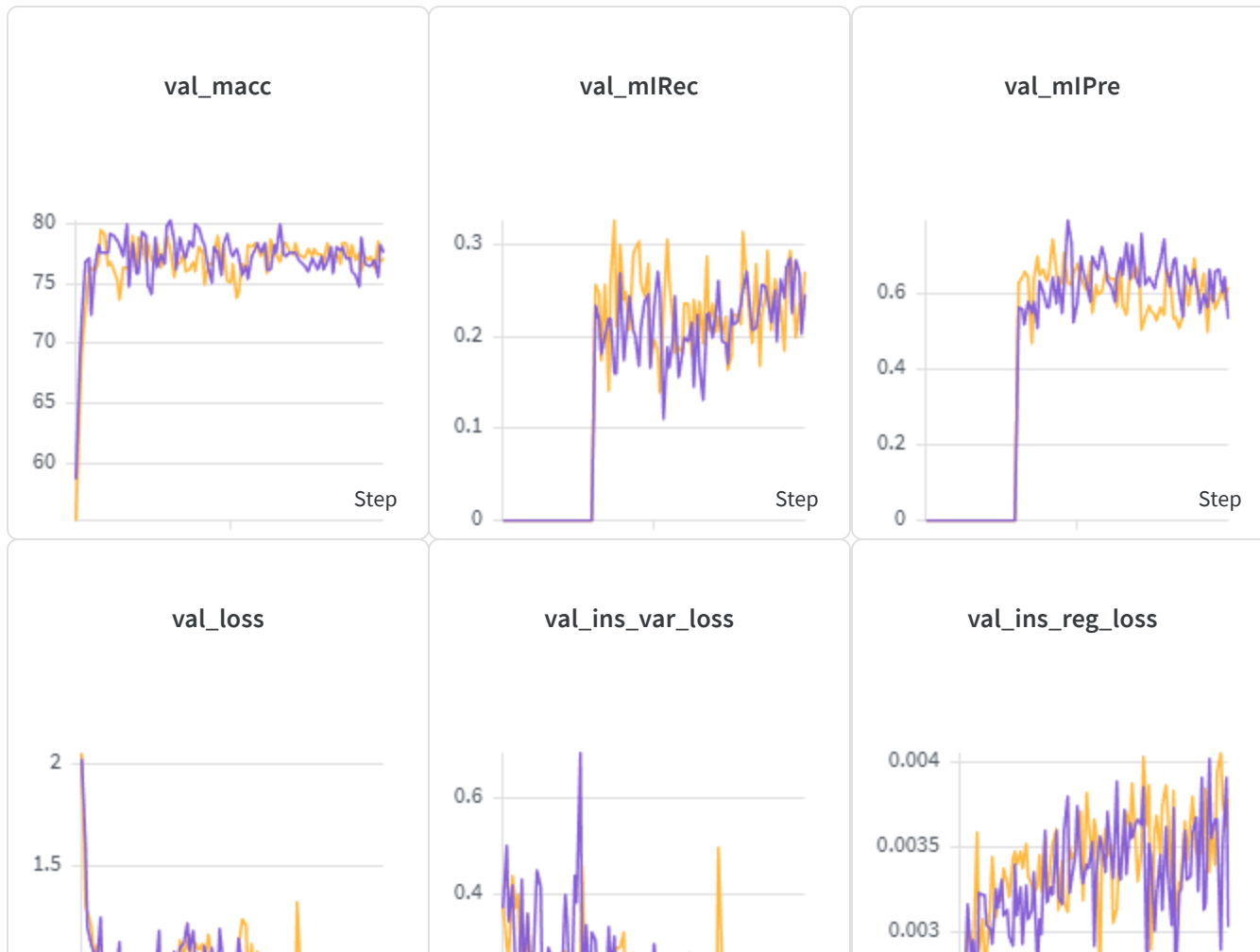
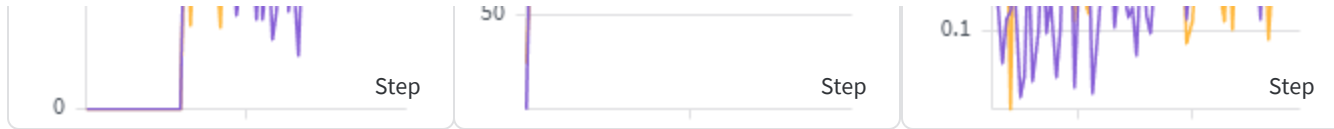
TorchSparse was 1.2× slower on the backbone and 2.1× slower once the scorer ran — the opposite of the expectation that motivated trying it. The cause is the shape of the workload rather than a defect in either library. The published claim that TorchSparse is faster benchmarks it against an older MinkowskiEngine on large, spatially coherent scenes; here the hot path is the opposite, as the scorer receives hundreds of small, spatially *disjoint* clusters in a freshly built sparse tensor on every iteration. That is close to the worst case for the hash-based lookups TorchSparse relies on, and the per-cluster matrix multiplications are too small to hide the cost of launching them.

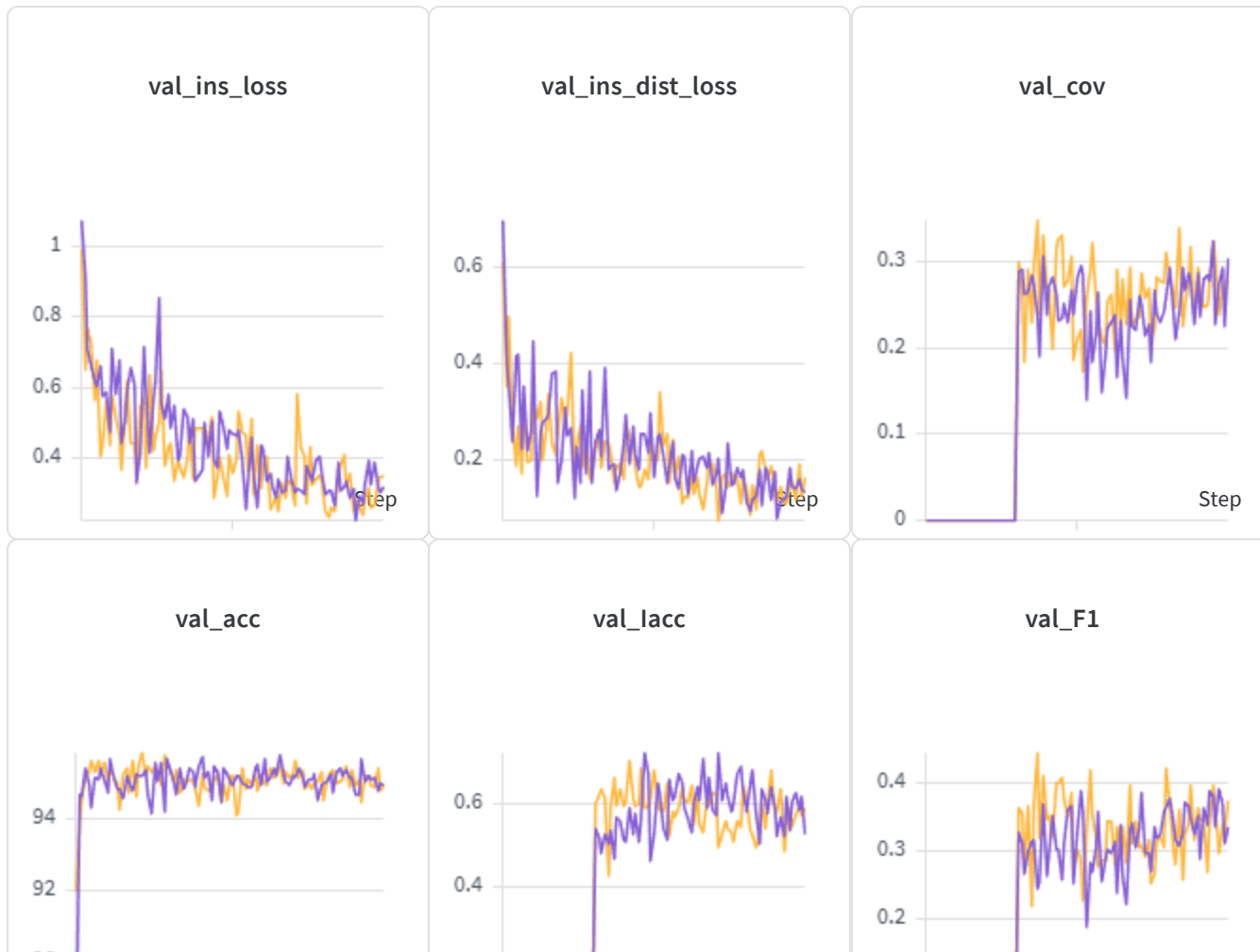
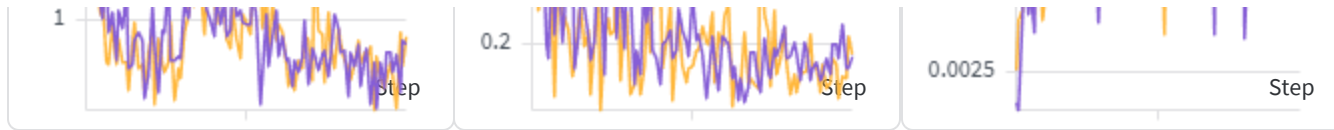
One caveat on reading the charts: the figures logged during training are running averages over subsampled cylinders, which makes them right for watching *progress* and wrong for quoting as a *result*. Sections 2 and 3 use full-resolution evaluation instead.

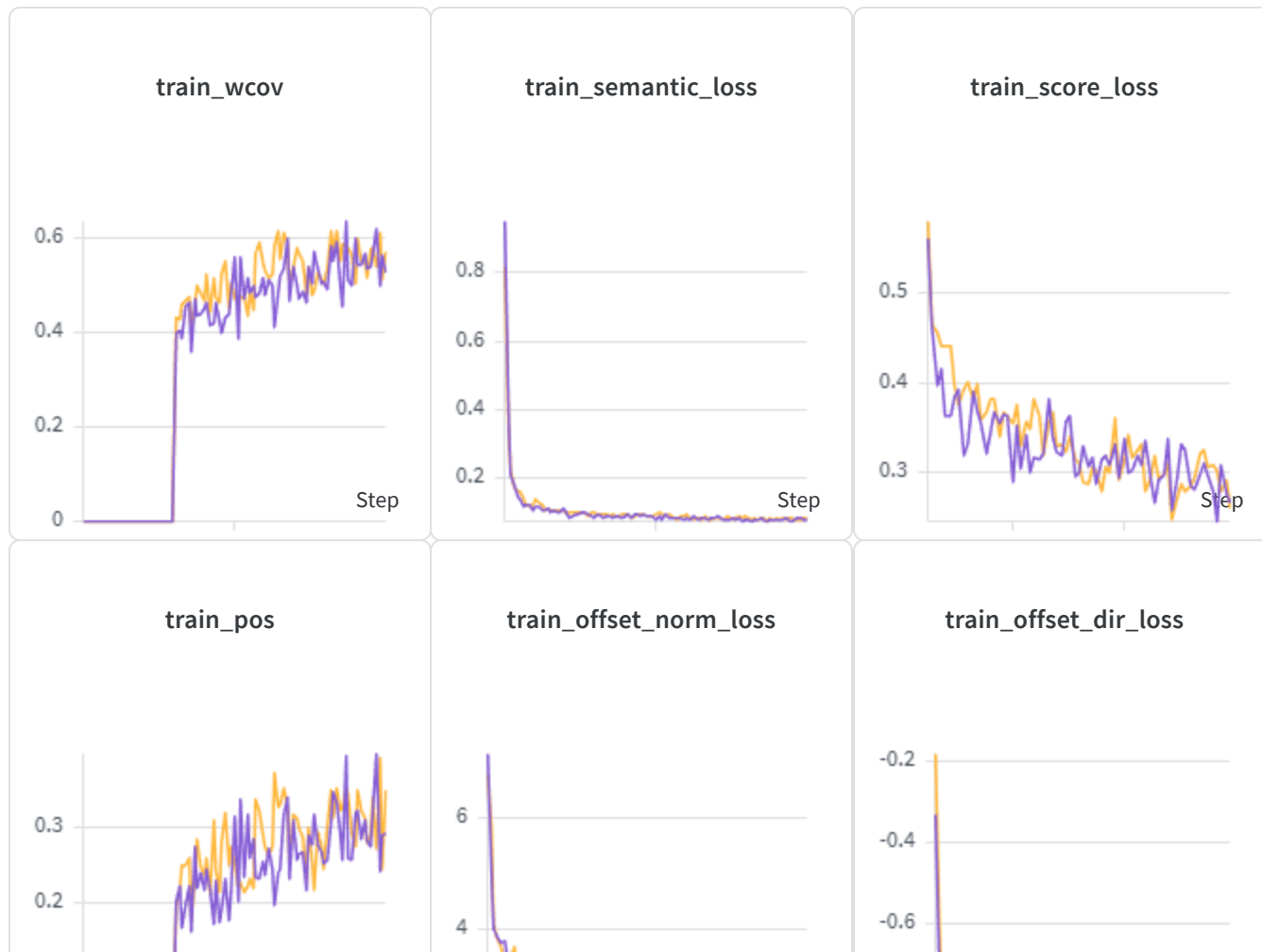
✓ Training charts

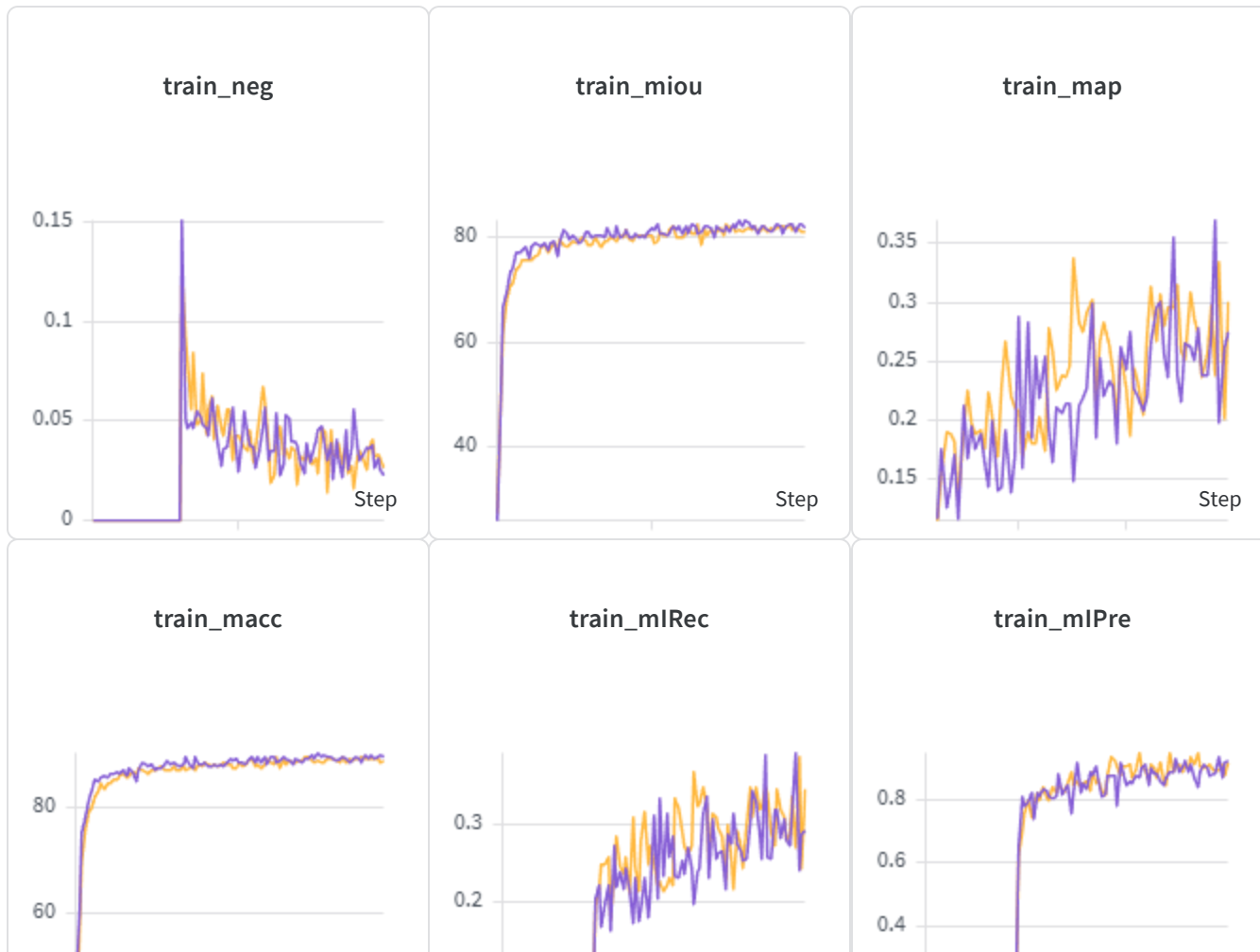
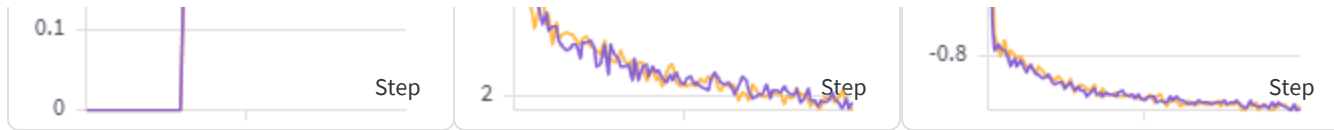


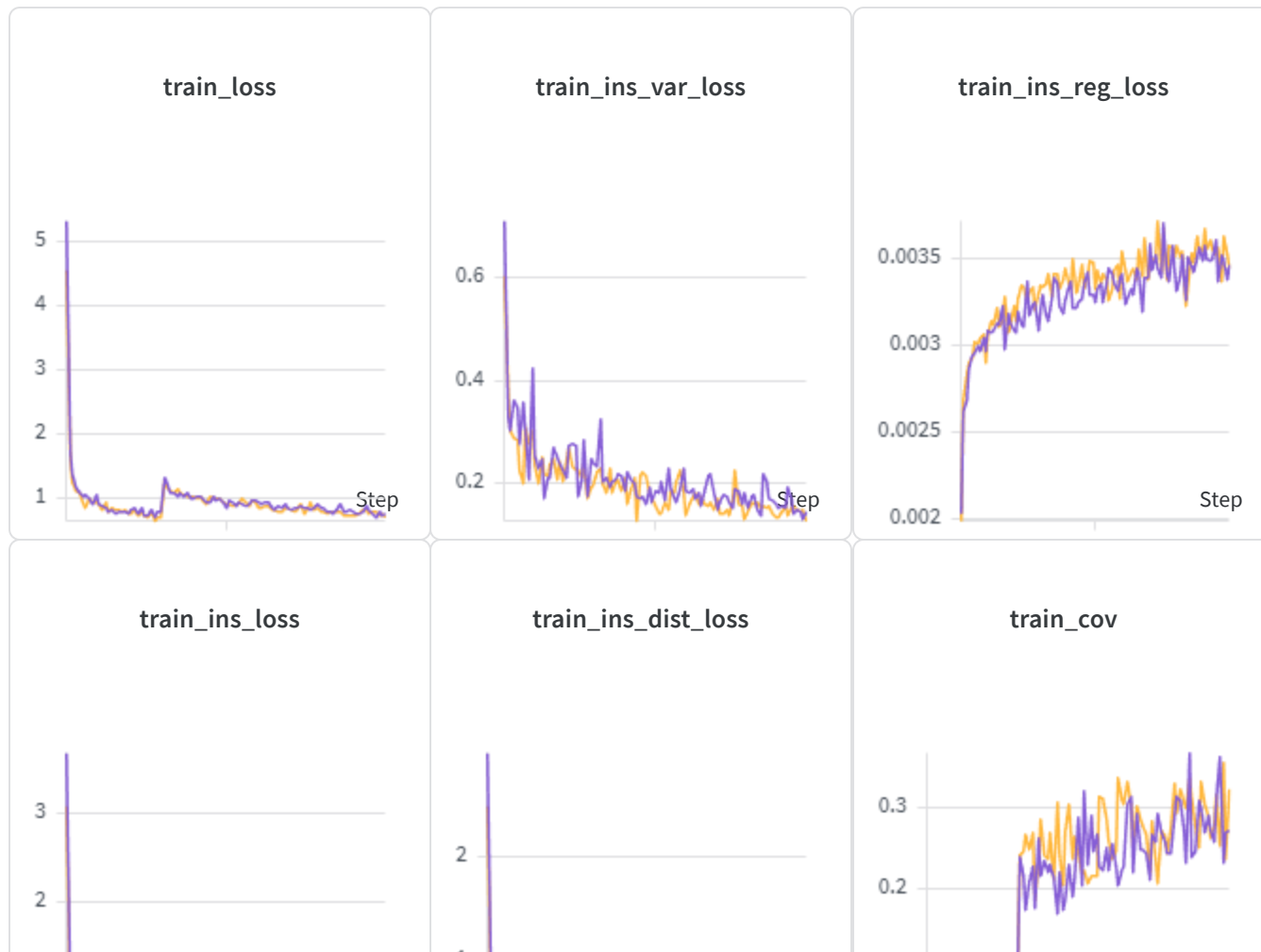


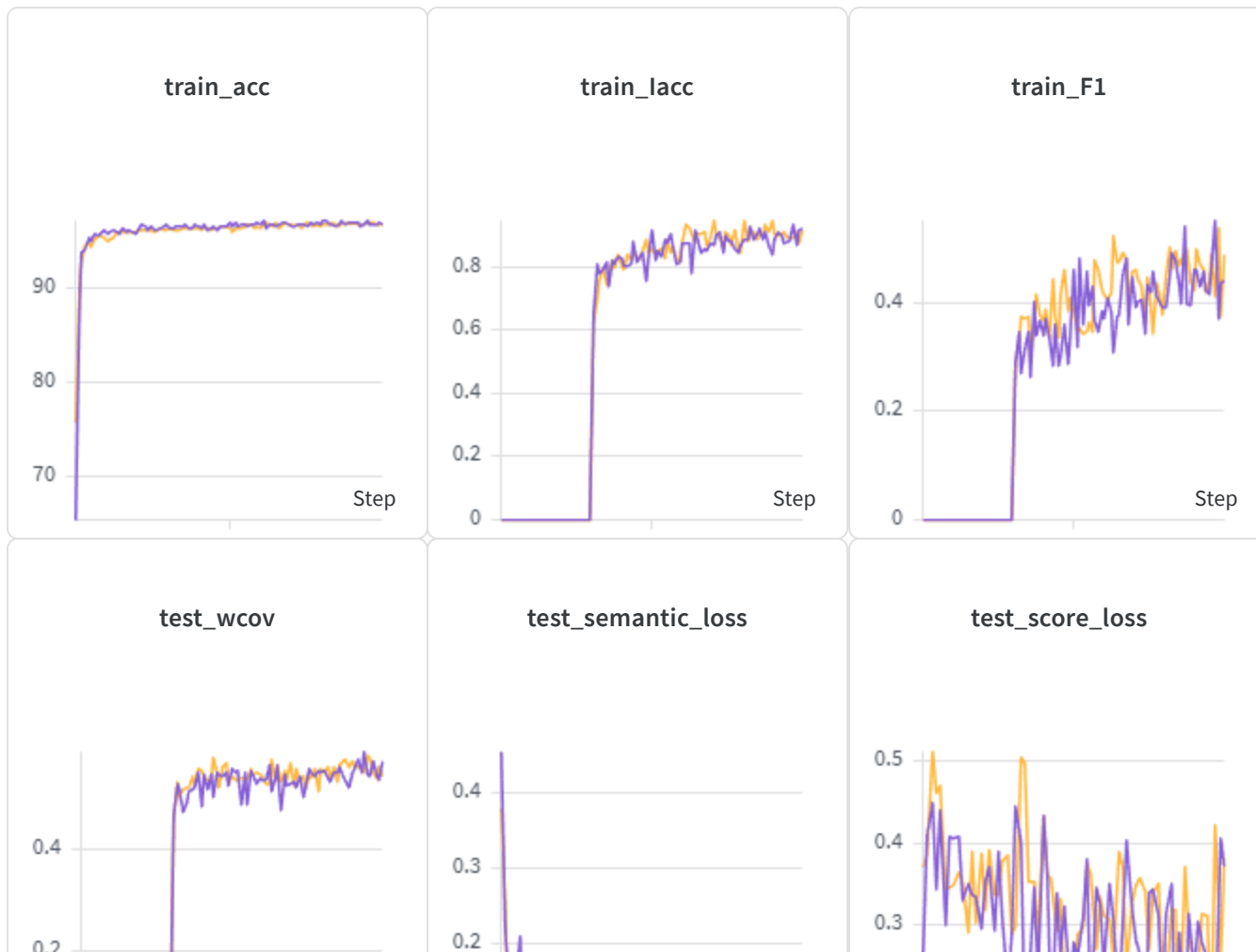




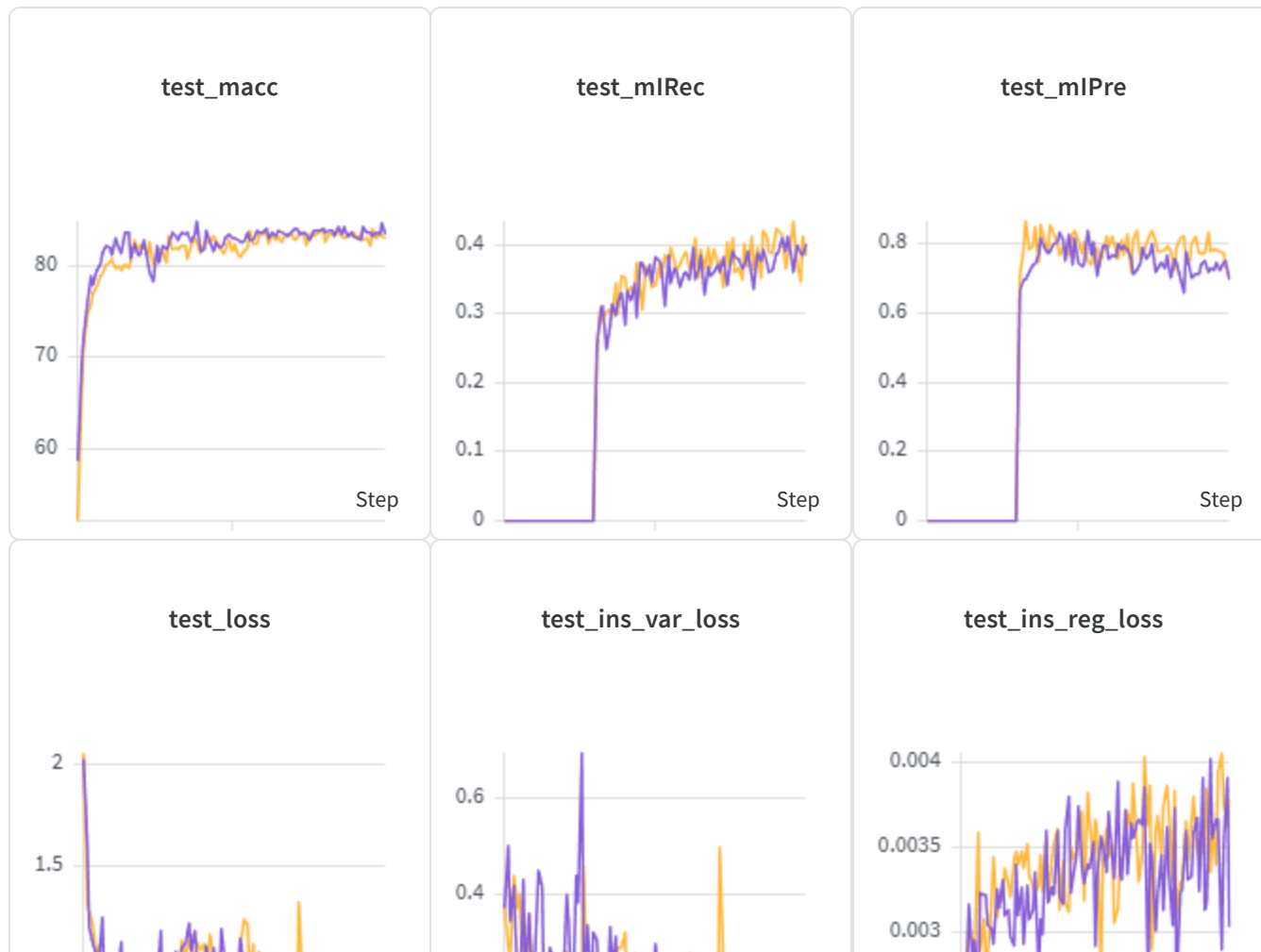


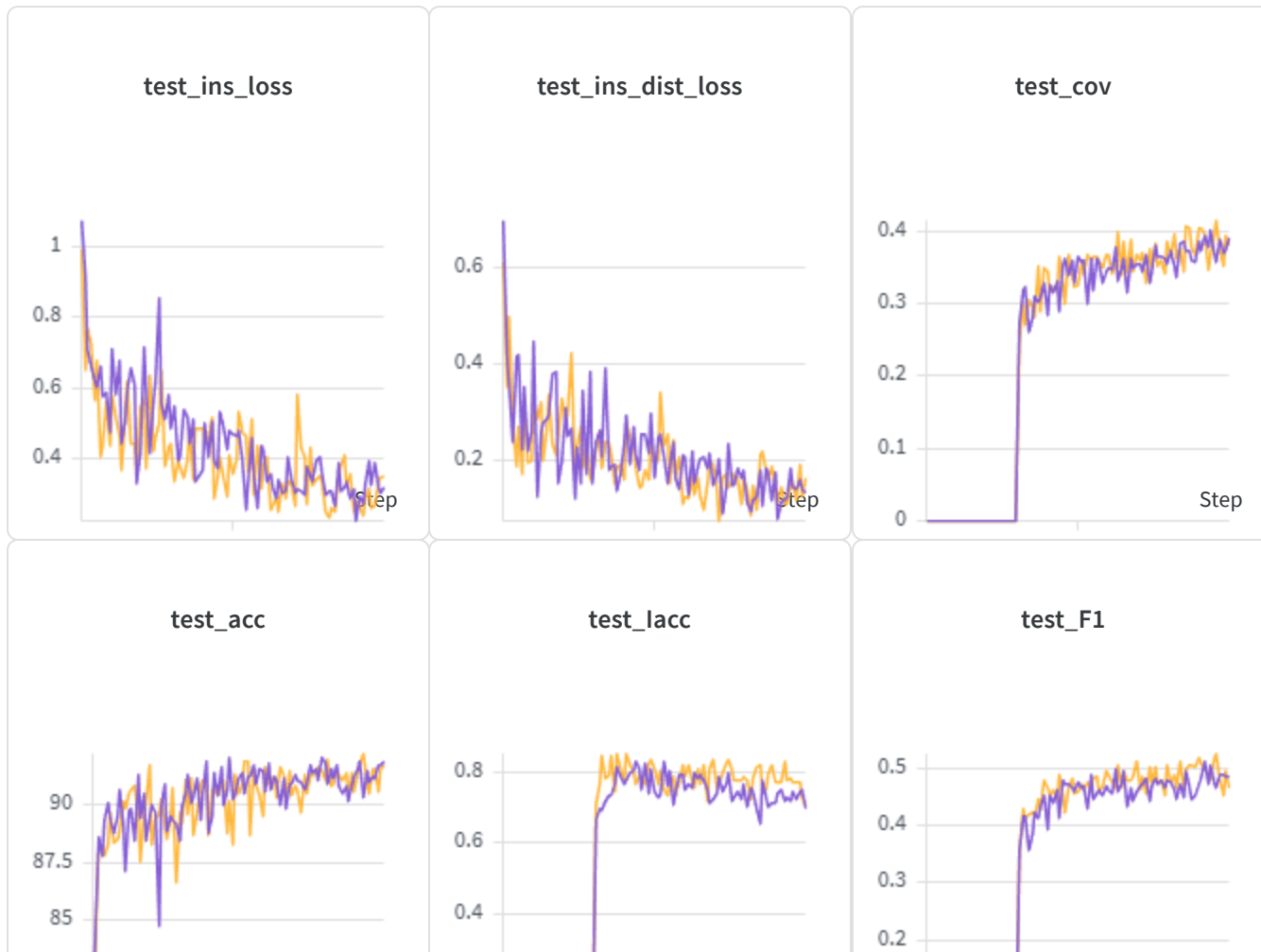
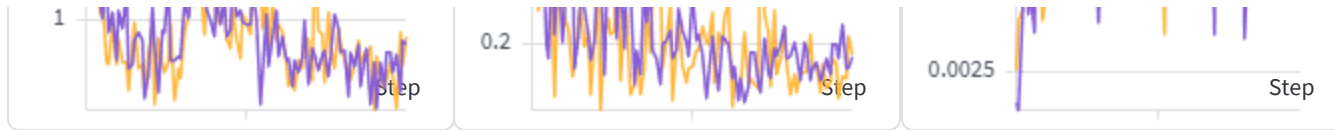




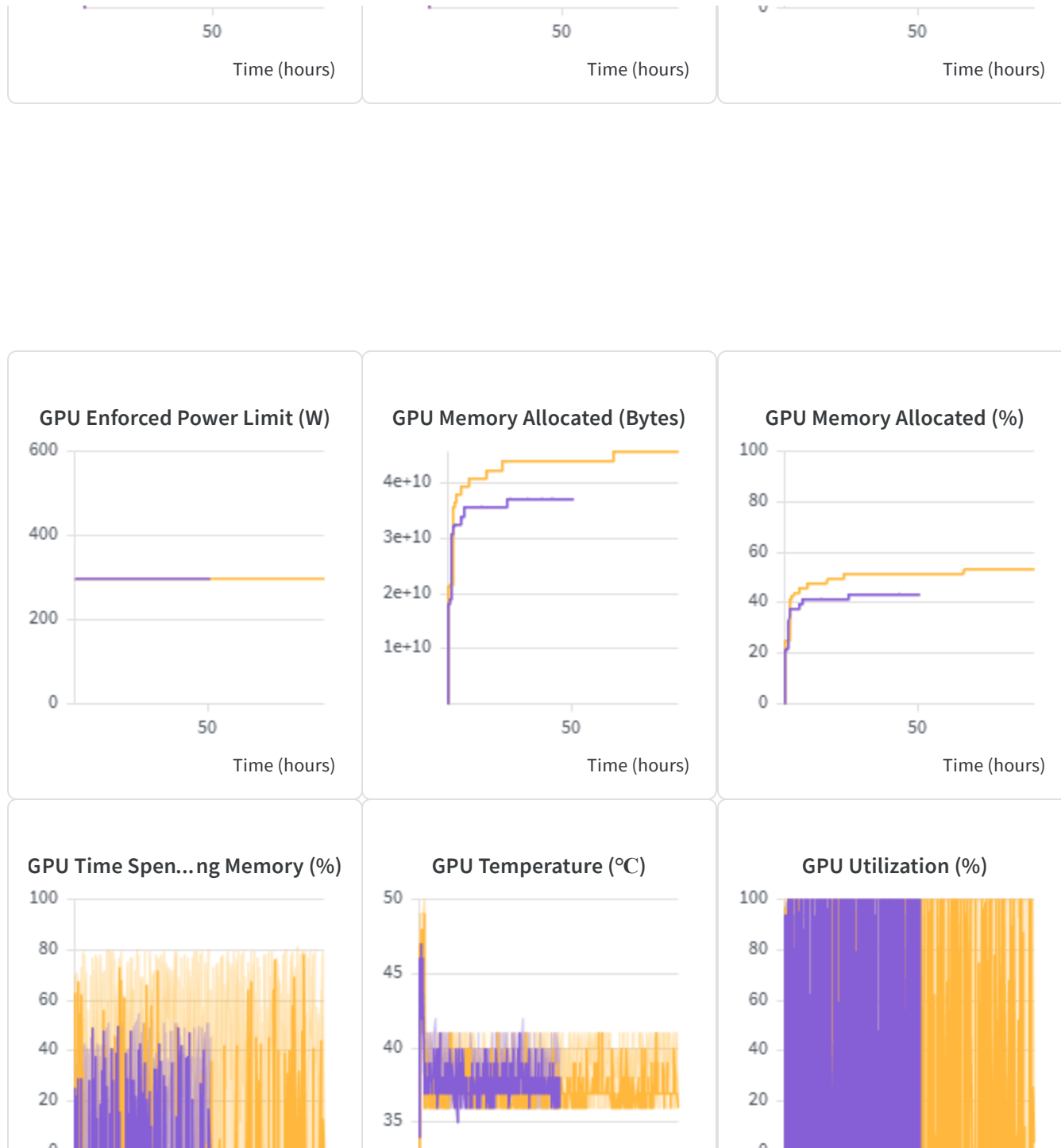


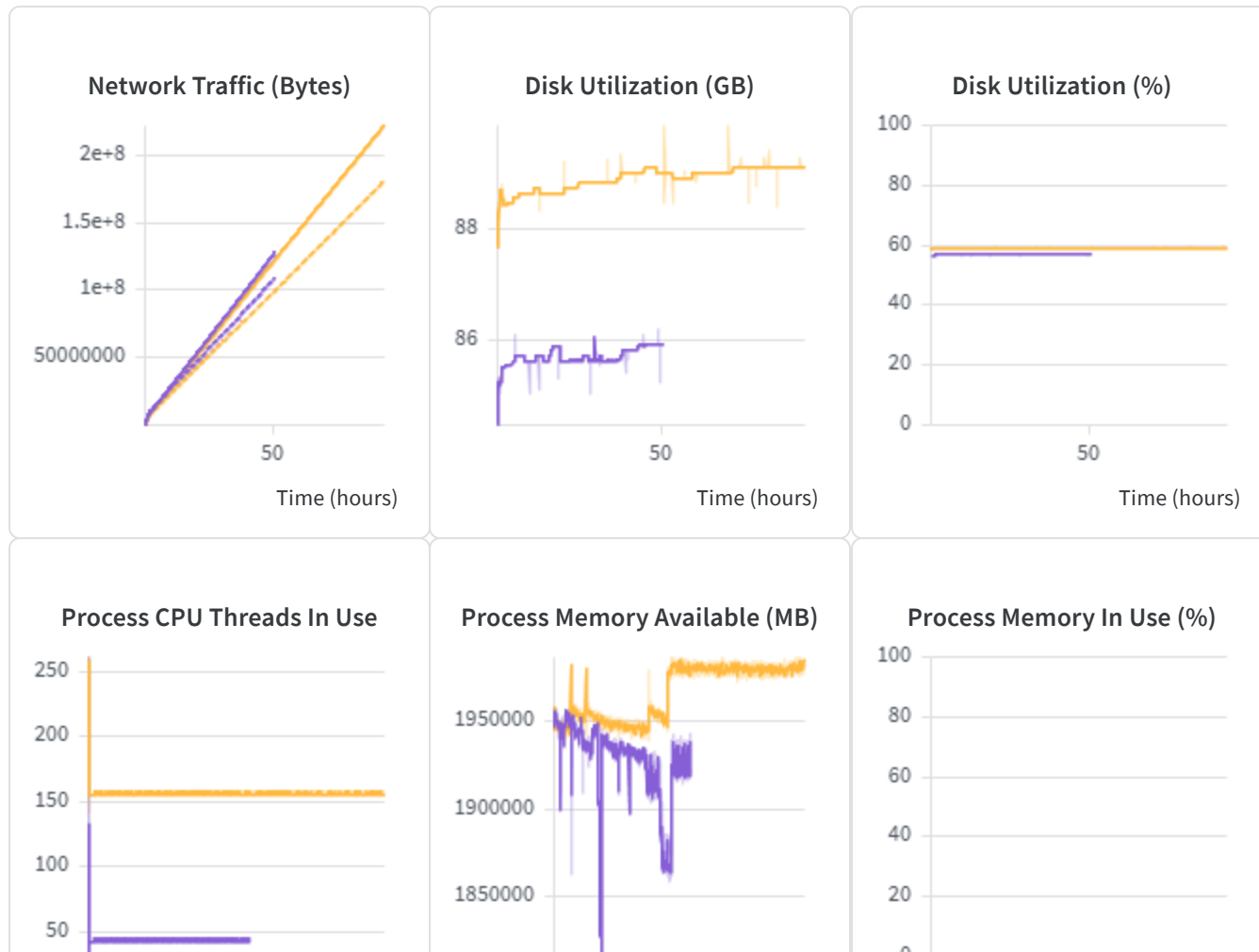
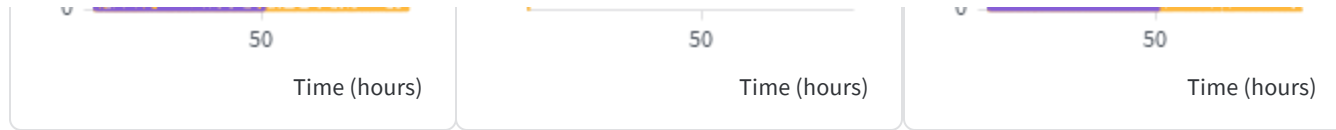


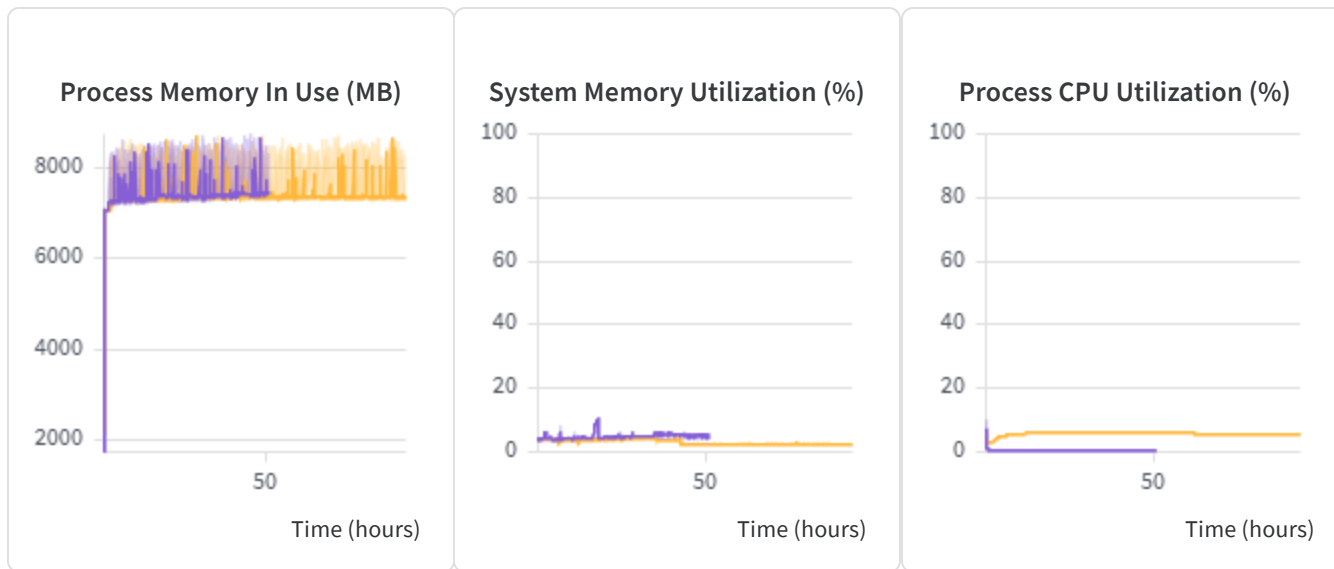
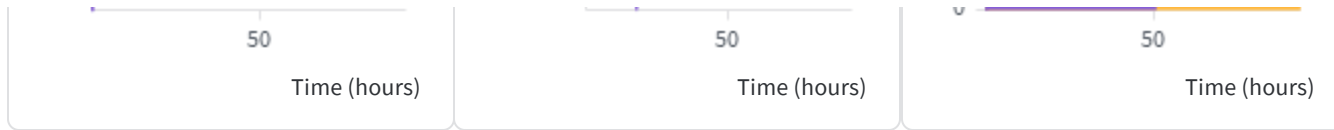












✓ MinkowskiEngine versus TorchSparse

Both models were evaluated on the two held-out test plots — 41,534,130 points — with the results pooled into a single confusion matrix and a single instance matching, rather than averaged per plot.

I used there `evaluate.py` and `evaluation_stats_FOR.py` scripts from ForAINet. The metrics have to be modified (4 semantic classes instead of 5).

Metric	MinkowskiEngine	TorchSparse 1.4
Overall accuracy	0.9048	0.8996
Mean accuracy	0.8594	0.8544
mIoU	0.7768	0.7662
mIoU excluding ground	0.7734	0.7602
Binary mIoU (tree / not-tree)	0.9603	0.9484
Instance precision	0.5055	0.4935
Instance recall	0.2383	0.1959
Instance F1	0.3239	0.2804
Coverage, unweighted / weighted	0.357 / 0.419	0.298 / 0.369
Segmentation quality, things	0.7321	0.7553
Panoptic quality, things	0.2372	0.2118

Per class, both models agree closely and rank the classes identically:

Class IoU	MinkowskiEngine	TorchSparse 1.4
low vegetation	0.7171	0.6913
ground	0.7872	0.7839
stem points	0.6952	0.6849
live branches	0.9079	0.9044

MinkowskiEngine leads on 25 of the 27 metrics compared. TorchSparse wins exactly one instance metric — *segmentation quality (things)* — and the interpretation is specific: it matched fewer trees, but outlined the ones it did match slightly more tightly. Since panoptic quality is recognition quality multiplied by segmentation quality, and the recognition gap is the larger of the two, MinkowskiEngine still comes out ahead overall.

These differences are **not evaluation noise**. Three evaluations of the same checkpoint produced byte-identical reports, for both backbones. One caveat applies: the numbers shift slightly depending on whether the run built the preprocessing cache or loaded an existing one — two regimes, each perfectly reproducible. Every figure above comes from cache-loading runs for both models, so they are directly comparable.

The full set of tables — every metric the evaluation emits, and a second grouping that also folds in the validation plot — is given in the detailed backbone comparison.

Conclusion on the backbone swap: it changed nothing that matters and cost speed. The original backbone stays the default.

✓ Comparison with the published results

The ForAINet paper reports its configurations in Table 4. I trained with the paper's **basic setting**, so that is the row to compare against.

The two evaluations use different *names* for the same quantities, and it is worth showing that they really are the same before putting them in one table. The paper reports completeness 79.3 % and commission error 21.2 %, implying a precision of 78.8 %; combining those as an F1 score gives 79.05 %, which is exactly the 79.0 the paper reports. So *completeness* is recall, *commission error* is one minus precision, and *F-score* is F1 — the same three numbers my evaluation prints.

Metric (%)	ForAINet paper (basic setting)	This work — MinkowskiEngine	This work — TorchSparse
Semantic — overall accuracy	92.6	90.5	90.0
Semantic — mean accuracy	81.2	85.9	85.4
Semantic — mIoU	73.0	77.7	76.6
Instance — completeness (recall)	79.3	23.8	19.6
Instance — omission error	20.8	76.2	80.4
Instance — commission error	21.2	49.5	50.6
Instance — F-score	79.0	32.4	28.0
Coverage	77.0	35.7 / 41.9	29.8 / 36.9

Bold marks the better value; for the two *error* rows, lower is better. Coverage is given as unweighted / weighted, since my evaluation distinguishes the two where the paper reports a single figure. For context, the paper's best configuration (+ TreeMix) reaches an F-score of 85.1 — I compare against the basic setting because that is what I trained.

Four things follow.

The semantic half transferred, and then some. An mIoU of 77.7 against the paper's 73.0, and mean accuracy of 85.9 against 81.2, say that the network learned to label terrestrial LiDAR points at least as well as it labels airborne ones. This deserves one honest caveat: my mIoU averages over **four** classes and theirs over five, and removing a class changes the denominator. "Comparable" is the defensible word; "better" is not.

The instance half did not transfer. An F-score of 32.4 against 79.0, and coverage of roughly half. This is the central finding of the project, and it is a result rather than a defect — the model was applied outside the conditions it was designed for, and the part that depends most on those conditions is the part that broke.

The most likely cause is the sensor, not the code. ForAINet's instance branch predicts, for every point, an offset vector toward its own tree's stem, clusters the shifted points, and scores the resulting clusters. That geometry is natural for airborne scanning, where a crown is well sampled from above and its stem lies directly beneath it. A terrestrial scan inverts it: stems are densely sampled, crowns are occluded and interleaved, and adjacent trees genuinely touch. One observation points the same way — the training-time tracker reported test F1 around 0.47–0.48 over subsampled cylinders against roughly 0.32 once full-resolution blocks are merged, placing part of the loss in the merge step rather than in the network.

Label provenance is a second candidate, and I cannot separate it from the first. SegmentedForests supplies semantic labels only, so the tree identifiers the instance branch trains on and is scored against were generated by 3DFin rather than delineated by hand. The paper's instance figures are measured against manual annotation. An unknown share of the gap is therefore the difficulty of the data, and an unknown share is the quality of the labels; telling them apart would need a manually delineated subset to test against, which this project does not have. The semantic comparison above is not affected — those labels are the dataset's own.

These are inferences, not measurements: I did not re-run ForAINet on FOR-Instance, so the paper's numbers and mine were produced on different data by different people.

✓ The six biggest challenges

1. The label scheme had to be designed, and it was not one-to-one. Sixteen source classes had to become four, and one of the framework's own classes had no counterpart in the new data at all. My first check, on a single plot, looked deceptively tidy — individual plots use very different subsets of the sixteen values, so only a check against all fourteen

revealed the real vocabulary. I therefore made the mapping strict: an unmapped value stops the run instead of defaulting to something plausible.

2. A silent convention mismatch destroyed the entire instance task. The instance identifiers produced by 3DFin follow the opposite convention to ForAINet's, marking ground points with the nearest tree's id. ForAINet's own filter then rejected every contaminated instance — zero of forty-two on the plot where I found it. Nothing crashed, nothing warned; training would have completed normally and learned instance segmentation from nothing at all. Fixing it meant clearing the id on 46.6 % of all points.

3. Data volume dictated the architecture of the pipeline. 4.4 GB of input expands to 46 GB of intermediates and 29 GB of uncompressed point clouds — the format the model reads costs exactly 37 bytes per point. Processing plot by plot and deleting each intermediate as soon as it was consumed brought peak disk use from 108.6 GB to 34.3 GB, and exporting compressed cut the upload from 29.2 GB to 4.3 GB. Concurrency had to be bounded by memory rather than by CPU count, since a single plot can need nearly 40 GB of RAM.

4. The development machine and the container disagreed, and only the container's opinion mattered. A YAML feature that is perfectly valid under the configuration library installed locally is a parse error under the older version pinned inside the training image. Because the failure happens while *reading* the config, both backbones died with a message that named neither of them. The same shape of problem appeared twice more: the image's LiDAR reader had no backend for the compressed format, and its NumPy version had removed functions the framework still called. Every local check had passed.

5. One configuration value hid three separate crashes behind thirty epochs of paid GPU time. Because the instance branch is gated until epoch 31, a removed NumPy alias and two distinct TorchSparse failures each surfaced only after roughly seven hours of A100 time. This is what made

me build a pre-flight gate that runs the full chain cheaply before any long run — and that gate itself once reported a **false pass**, because a piped command returned the exit status of the wrong process. A gate that green-lights a broken run is worse than no gate, so I changed it to require positive evidence from the log rather than an exit code.

6. Making the comparison trustworthy was a project of its own. The script that produces the final pooled report was still written for five classes and, as shipped, could not run at all — I had to repair five separate faults before it produced a number. Evaluation turned out to be perfectly deterministic, but with two distinct regimes depending on whether a preprocessing cache already existed, which meant I could only compare runs from the same regime. And a claim I had made earlier, that instance metrics varied between runs, I had to retract once I actually measured it.

✓ Data availability

I created a public GitHub [repository](#) available under MIT license.

https://wandb.ai/aifor/FOR-dataset/reports/Minkowski_eng-vs-TorchSparse-1-4-using-SegmentedForests-modified-dataset---VmldzoxNzk2MzE5Ng